

CiT Backend Engineering Track

Week 6: Databases & SQL Foundations

Relational Databases, Tables, SQL Queries & Connecting Java with JDBC

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 6
Instructor	Jim & Anderson
Duration	6 months 24 weeks
Tech Stack	Java 21, PostgreSQL, DBeaver, IntelliJ IDEA

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we learn to store, structure, and work with the knowledge and information You allow us to handle. Give us the diligence to treat data with care, the wisdom to organise our work well, and the integrity to safeguard what belongs to others. Let us never grow careless with what is entrusted to us.

As Proverbs 10:14 reminds us: "Wise people store up knowledge." Lord, make us wise builders — thoughtful in what we keep, faithful in what we manage, and humble enough to keep learning.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,

Amen.

Scripture for the Week

*"Wise people store up knowledge, but the mouth of the foolish is near destruction."
— Proverbs 10:14 (NKJV)*

Table of Contents

1. Welcome to Week 6
2. Why Databases? From Memory to Permanent Storage
3. Relational Databases & Tables
4. Designing a Table — Data Types, Keys & Constraints
5. SQL Part 1 — Creating Tables & Inserting Data
6. SQL Part 2 — Querying Data with SELECT
7. SQL Part 3 — Updating, Deleting & Joining Tables
8. Connecting Java to a Database with JDBC
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 6

Over the last three weeks you learned the Java language — variables, control flow, methods, object-oriented design, collections, and exception handling. You can now build real programs. But every one of those programs has shared a hidden weakness: the moment it stops running, all its data disappears.

Your Student Grade Management System holds its students in a list in memory. Close the program, and every student is gone. Real systems cannot work this way. A bank cannot forget your balance overnight. A school cannot lose its students when the power goes out. This week we solve that problem with databases.

A database stores data permanently and safely, separate from your program. To talk to it, you use a language called SQL. By the end of today you will create tables, store and retrieve data with SQL, and connect a Java program to a real database — giving your Student Grade Management System a permanent memory.

What to Expect in Week 6

By the end of this session, you will be able to:

- Explain why programs need databases and what "persistence" means
- Describe a relational database in terms of tables, rows, and columns
- Design a table with sensible data types, a primary key, and constraints
- Write SQL to create tables and insert data (CREATE TABLE, INSERT)
- Query data with SELECT, WHERE, and ORDER BY
- Modify and relate data with UPDATE, DELETE, and JOIN
- Connect a Java program to a database using JDBC

2. Why Databases? From Memory to Permanent Storage

Before any syntax, understand the problem a database solves. This is the "why" that makes everything else make sense.

2.1 Memory vs Permanent Storage

Your programs so far have stored data in memory (RAM). Memory is fast, but it is temporary — it is wiped clean the moment the program ends or the computer restarts. It is like writing on a whiteboard: perfect for working things out, but erased at the end of the day.

A database stores data on disk, permanently. It is like writing in a ledger book: it stays there tomorrow, next week, and next year, no matter how many times the program restarts. We call data that survives like this persistent data, and the act of saving it persistence.

In Memory (a List/Map)	In a Database
Lost when the program stops	Survives restarts, crashes, and power cuts
Only your one running program can see it	Many programs and users can share it safely
Limited by the size of RAM	Can hold millions of records on disk
No protection if two users change it at once	Manages many simultaneous users safely

2.2 What Is a Database, Really?

A database is an organised collection of data stored so it can be easily added to, retrieved, updated, and managed. The software that manages it is called a Database Management System (DBMS).

In this course we use PostgreSQL — a powerful, free, professional-grade database used by companies worldwide, including many here in Uganda. The DBMS runs as a service on a computer (a "server"), and your programs connect to it to store and fetch data.

A Familiar Analogy – The School Records Office

Imagine a school records office. It has filing cabinets (the database), each drawer holds a particular kind of record like "Students" or "Fees" (the tables), and inside are individual record cards (the rows). The records clerk who fetches, files, and updates cards on request is the DBMS. You never touch the cabinets directly – you ask the clerk, in a precise language. That language is SQL.

2.3 What Is SQL?

SQL stands for Structured Query Language. It is the standard language for talking to relational databases. You write short, English-like commands – "give me all students with a GPA above 3.5", "add this new student", "delete this record" – and the database carries them out.

SQL is not like Java. You do not write loops or tell the database how to find the data step by step. You simply describe what you want, and the database figures out how to get it. This is called a declarative language.

3. Relational Databases & Tables

The kind of database we use is a relational database. The name comes from the way it organises data into tables that can relate to one another. Almost every business system in the world is built on this model.

3.1 Tables, Rows & Columns

A table stores one type of thing — students, courses, payments. If you have used a spreadsheet, you already understand the shape of a table.

Term	Plain English
Table	A grid that holds one kind of record. Example: a "students" table holding all students.
Column	One piece of information every record has — e.g. name, age, gpa. Also called a field. Each column has a fixed data type.
Row	One complete record — one single student, with a value in every column. Also called a record.

3.2 A "students" Table — Visualised

Here is what a students table looks like. The top row is the column names; each row below is one student.

id	name	gpa
1	Alice Nakato	3.75
2	Bob Okello	3.10
3	Carol Atim	3.90

3.3 Why Multiple Related Tables?

You might ask: why not keep everything in one giant table? Because real data has relationships, and cramming it into one table causes duplication and errors.

Imagine storing each student's courses in the students table. A student taking five courses would need their name, age, and registration number repeated five times. Change their name once and you must change it in five places — and if you miss one, the data is now inconsistent.

Instead, relational databases split data into focused tables — one for students, one for courses — and link them. We store each fact once and connect the tables when we need to combine them. You will see exactly how with JOIN in Section 7.

The Golden Rule of Relational Design

Each table should describe one kind of thing, and each fact should be stored in exactly one place. When you feel tempted to repeat the same information in many rows, that is a sign the data belongs in its own table.

4. Designing a Table — Data Types, Keys & Constraints

Before storing data, you must define the shape of your table: what columns it has, what type of data each holds, and what rules keep the data clean. Good design here prevents a thousand bugs later.

4.1 Column Data Types

Just like Java variables, every column has a data type. The database rejects any value that does not fit. Here are the types you will use most in PostgreSQL.

SQL Type	Holds	Example
INT	Whole numbers — ages, counts, IDs	age INT
SERIAL	A whole number that auto-increments — perfect for IDs	id SERIAL
VARCHAR(n)	Text up to n characters — names, emails	name VARCHAR(100)
TEXT	Text of any length — descriptions, notes	bio TEXT
NUMERIC(p,s)	Exact decimals — money, GPA. p digits, s after the point	gpa NUMERIC(3,2)
BOOLEAN	true or false — flags like is_active	is_active BOOLEAN
DATE	A calendar date	enrolled_on DATE

4.2 The Primary Key

Every table needs a way to identify each row uniquely. That column is called the primary key. No two rows may share the same primary key, and it can never be empty.

Think of it like a national ID number — it belongs to exactly one person and reliably tells them apart, even if two people share the same name. We usually use an auto-incrementing id column of type SERIAL as the primary key.

4.3 Constraints — Rules That Protect Your Data

A constraint is a rule the database enforces on a column. If a value breaks the rule, the database refuses to store it. Constraints are how a database protects itself from bad data.

Constraint	What It Enforces
PRIMARY KEY	This column uniquely identifies each row; unique and never empty.
NOT NULL	This column must always have a value — it cannot be left empty.
UNIQUE	No two rows may have the same value here — e.g. registration numbers, emails.
DEFAULT	If no value is given, use this one automatically.
CHECK	The value must satisfy a condition — e.g. a GPA between 0 and 4.

Why Constraints Matter

Without a NOT NULL rule, a student could be saved with no name. Without UNIQUE on registration numbers, two students could share one. Constraints let the database itself guarantee your data stays sensible — even if a buggy program tries to break it.

5. SQL Part 1 — Creating Tables & Inserting Data

Now we write real SQL. SQL commands fall into groups; we start with creating a table and adding rows. You will run these in DBeaver, a free visual tool for working with databases.

5.1 CREATE TABLE

The CREATE TABLE command defines a new table: its name, its columns, each column's type, and any constraints. Notice how readable it is.

```
CREATE TABLE students (  
  id      SERIAL      PRIMARY KEY,  
  name    VARCHAR(100) NOT NULL,  
  reg_number VARCHAR(20) UNIQUE NOT NULL,  
  age     INT,  
  gpa     NUMERIC(3,2) DEFAULT 0.0,  
  is_active BOOLEAN    DEFAULT TRUE  
);
```

Read it aloud: "create a table called students, with an auto-numbered id as its primary key, a name that cannot be empty, a registration number that must be unique and present, an age, a gpa that defaults to 0, and an active flag that defaults to true."

5.2 INSERT — Adding Rows

The INSERT INTO command adds a new row. You list the columns you are filling, then the matching values. You do not provide the id — SERIAL fills it automatically.

```
INSERT INTO students (name, reg_number, age, gpa)  
VALUES ('Alice Nakato', '2024001', 22, 3.75);  
  
INSERT INTO students (name, reg_number, age, gpa)  
VALUES ('Bob Okello', '2024002', 24, 3.10);
```

Text Goes in Single Quotes

In SQL, text values are wrapped in single quotes: 'Alice Nakato'. Numbers are written plain: 22. This is the opposite of nothing you have seen — just remember: strings get single quotes, numbers do not.

Exercise 1: Create a courses Table

Write the SQL to create a courses table with these columns:

- id — an auto-incrementing primary key
- title — text up to 100 characters, required
- code — text up to 10 characters, required and unique
- credit_units — a whole number

Then write two INSERT statements to add two real courses of your choice.

6. SQL Part 2 — Querying Data with SELECT

Storing data is only half the job. The real power of a database is getting exactly the data you want back out. That is the job of the SELECT command — the one you will use more than any other.

6.1 SELECT — The Basics

A SELECT statement asks the database for data. You name the columns you want and the table to read from.

```
SELECT * FROM students;      -- every column, every row
SELECT name, gpa FROM students; -- only these two columns
```

The * means "all columns". In real code, prefer naming the columns you actually need — it is clearer and faster.

6.2 WHERE — Filtering Rows

The WHERE clause keeps only the rows that match a condition — the database equivalent of an if.

```
SELECT name, gpa FROM students WHERE gpa >= 3.5;

SELECT * FROM students WHERE reg_number = '2024001';

SELECT name FROM students WHERE age > 21 AND is_active = TRUE;
```

Operator	Meaning
= <> < > <= >=	Compare values (note: SQL uses a single = for equality, and <> for not-equal)
AND OR NOT	Combine several conditions
BETWEEN a AND b	Within a range, inclusive
LIKE 'A%'	Pattern match — names starting with A (% means "any characters")
IN (...)	Matches any value in a list

6.3 ORDER BY — Sorting Results

The ORDER BY clause sorts the results. Add DESC for descending (highest first) or ASC for ascending (the default).

```
SELECT name, gpa FROM students ORDER BY gpa DESC; -- best students first
SELECT name FROM students ORDER BY name ASC;      -- alphabetical
```

Exercise 2: Query the students Table

Using the students table from this week, write SQL for each request:

- All students, showing only their name and registration number
- Students with a GPA of 3.5 or higher, best first
- Students older than 21 who are still active
- Students whose name starts with the letter "A" (use LIKE)

7. SQL Part 3 — Updating, Deleting & Joining Tables

Data changes over time. Students update their details, records get removed, and we constantly need to combine related tables. This section completes your core SQL toolkit.

7.1 UPDATE — Changing Existing Rows

The UPDATE command changes data in rows that match a condition. The WHERE clause decides which rows.

```
UPDATE students SET gpa = 3.80 WHERE reg_number = '2024001';
```

```
UPDATE students SET is_active = FALSE WHERE age > 30;
```

NEVER Forget the WHERE Clause

An UPDATE or DELETE with no WHERE affects EVERY row in the table. UPDATE students SET gpa = 0; would wipe every student's GPA to zero. Always write your WHERE first, and double-check it before running.

7.2 DELETE — Removing Rows

The DELETE command removes rows that match a condition.

```
DELETE FROM students WHERE reg_number = '2024002';
```

7.3 Relating Tables — Foreign Keys

Recall from Section 3 that we split data into focused tables. To link them, one table holds a foreign key — a column that points to the primary key of another table.

```
CREATE TABLE enrollments (  
  id SERIAL PRIMARY KEY,  
  student_id INT REFERENCES students(id),  
  course_id INT REFERENCES courses(id),  
  grade VARCHAR(2)  
);
```

Here student_id is a foreign key: it stores the id of a student. This is how one enrollment row "belongs to" one student and one course, without repeating their details.

7.4 JOIN — Combining Tables in a Query

A JOIN combines rows from two tables by matching a foreign key to a primary key. This is how you answer questions that span tables, like "which courses is each student taking?"

```
SELECT students.name, courses.title, enrollments.grade  
FROM enrollments  
JOIN students ON enrollments.student_id = students.id  
JOIN courses ON enrollments.course_id = courses.id;
```

This reads each enrollment, looks up the matching student and course, and returns one combined row per enrollment — the student's name, the course title, and the grade, all together.

8. Connecting Java to a Database with JDBC

You now know SQL, and you know Java. JDBC is the bridge between them. It lets a Java program connect to a database, send SQL, and read the results back as Java objects. This is what gives your Student Grade Management System a permanent memory.

8.1 What JDBC Is

JDBC stands for Java Database Connectivity. It is a standard set of Java classes for talking to databases. You add a small library called a driver for your specific database (PostgreSQL), and JDBC handles the conversation.

JDBC Class	Its Job
Connection	The open line to the database. You open it, use it, and close it.
PreparedStatement	A safe SQL statement with placeholders for values you fill in.
ResultSet	The table of rows returned by a SELECT — you read it row by row.

8.2 A Complete Example — Reading Students

This program connects to the database, asks for students with a high GPA, and prints them. The try-with-resources you met in Week 5 ensures the connection closes automatically.

```
String url = "jdbc:postgresql://localhost:5432/cit_school";
String sql = "SELECT name, gpa FROM students WHERE gpa >= ?";

try (Connection conn = DriverManager.getConnection(url, "postgres", "secret");
    PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setDouble(1, 3.5);    // fill the ? with 3.5
    ResultSet rs = stmt.executeQuery();

    while (rs.next()) {        // step through each row
        String name = rs.getString("name");
        double gpa = rs.getDouble("gpa");
        System.out.println(name + " — " + gpa);
    }
}
```

```
}  
} catch (SQLException e) {  
    System.out.println("Database error: " + e.getMessage());  
}
```

8.3 Why the ? Placeholder Matters

Notice we used ? instead of pasting the value straight into the SQL text. This is a `PreparedStatement`, and it is essential.

SQL Injection – A Critical Security Lesson

If you build SQL by gluing user input into a string, an attacker can type SQL into your form and take over your database. This attack is called SQL injection, and it is one of the most common security holes in the world.

`PreparedStatement` with ? placeholders prevents it entirely – the database treats your values as data, never as commands. Rule: never concatenate user input into SQL. Always use placeholders.

Project Milestone – Persist Your Students

Extend the Student Grade Management System so it stores its data in PostgreSQL instead of an in-memory list:

1. Create the students table in your database
2. Write a method that inserts a new student using a `PreparedStatement`
3. Write a method that loads all students with a `SELECT` and returns them as a `List<Student>`
4. Confirm the data survives after you stop and restart the program

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Databases store data permanently (persistence); memory loses it when the program stops
- Relational databases organise data into tables of rows and columns, split by what each describes
- Column data types, primary keys, and constraints (NOT NULL, UNIQUE, DEFAULT, CHECK) keep data clean
- Creating tables and adding data with CREATE TABLE and INSERT
- Querying with SELECT, WHERE, and ORDER BY
- Changing and relating data with UPDATE, DELETE, foreign keys, and JOIN
- Connecting Java to a database with JDBC, and preventing SQL injection with PreparedStatement

9.2 Homework

5. Complete all exercises from today — save your SQL scripts and push them to your GitHub repository
6. Design and create three related tables — students, courses, and enrollments — then write a JOIN that lists every student with the courses they are enrolled in
7. Complete the project milestone: make your Student Grade Management System save and load students from PostgreSQL using JDBC
8. Preview Week 7: Read about what a REST API is and what the Spring framework does. Come with one question ready.

Coming Next — Week 7: Building APIs with Spring Boot

Jim & Anderson will show you how to turn your Java and database skills into a real web service. You will meet the Spring Boot framework and build your first REST API — the kind of backend that powers mobile apps and websites. Everything from Weeks 3 to 6 comes together into something the outside world can talk to.

Closing Reflection

"He who is faithful in what is least is faithful also in much; and he who is unjust in what is least is unjust also in much."

— Luke 16:10 (NKJV)

A database holds the records of real people — their money, their grades, their lives. Handle that data with the same care and honesty you would want shown to your own. Faithfulness in small, unseen details — a constraint here, a careful WHERE clause there — is the mark of an engineer who can be trusted with much.

Glossary of Terms

Term	Definition
Column	A named field in a table that holds one piece of information of a fixed type
Constraint	A rule the database enforces on a column, e.g. NOT NULL, UNIQUE, CHECK
Database	An organised collection of data stored permanently and managed by a DBMS
DBMS	Database Management System — the software that stores and serves data (e.g. PostgreSQL)
Foreign Key	A column that points to the primary key of another table, linking the two
JDBC	Java Database Connectivity — the standard Java API for talking to databases
JOIN	A SQL operation that combines rows from two tables by matching related columns
Persistence	Storing data so it survives after a program stops running
PreparedStatement	A safe, reusable SQL statement with ? placeholders that prevents SQL injection
Primary Key	A column whose value uniquely identifies each row; unique and never empty
Relational Database	A database that organises data into tables that can relate to one another
ResultSet	The set of rows returned by a SELECT query, read in Java row by row
Row	A single complete record in a table
SQL	Structured Query Language — the standard language for querying relational databases
Table	A grid of rows and columns that stores one kind of record